

Aplicaciones

Tema 5 · Cápsula: reducción de ruido por sustracción espectral

Jesús Chacón Sombría

Departamento de Arquitectura de Computadores y Automática
Universidad Complutense de Madrid

Grado en Ingeniería Electrónica de Comunicaciones
Curso 2026-2027

Restar el ruido en cada celda de la STFT

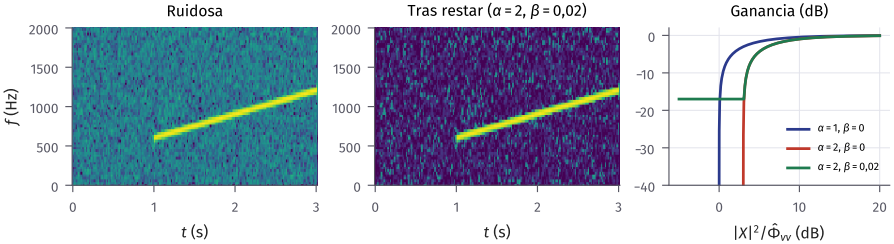
- Ruido aditivo, incorrelado con la señal y estacionario: en cada trama m y frecuencia k, $E\{|X[m,k]|^2\} \approx |S[m,k]|^2 + \Phi_{vv}[k]$.
- Se estima $\hat{\Phi}_{vv}[k]$ promediando $|X|^2$ en tramas solo de ruido (un silencio inicial) y se resta potencia:

$$|\hat{S}[m,k]|^2 = \max\left(|X[m,k]|^2 - \alpha\,\hat{\Phi}_{vv}[k],\ \beta|X[m,k]|^2\right)$$

- Se conserva la fase ruidosa, $\hat{S} = |\hat{S}|\,e^{j\angle X}$: el oído es poco sensible a la fase.
- Se vuelve al tiempo con la ISTFT (solapamiento y suma con una ventana que cumpla COLA).
- Equivale a una ganancia que cambia con m y k: $|G|^2 = \max\left(1 - \alpha\hat{\Phi}_{vv}/|X|^2, \beta\right)$.

▶ Herramienta interactiva: `T5-stft`

Ajuste: α, β y el ruido musical



- Ruido musical: $|X|^2$ fluctúa alrededor de Φ_{vv}; en las celdas donde por azar supera $\alpha\hat{\Phi}_{vv}$ queda un pico aislado que suena como un tono breve de frecuencia aleatoria.
- $\alpha > 1$ (sobresustracción, 2–5) elimina más picos a cambio de distorsionar la señal débil.
- β (suelo, 0,01–0,1) deja un ruido de fondo continuo que enmascara los picos aislados. Más β: menos ruido musical y más ruido residual.
- Longitud de trama: la de la STFT (20–40 ms en audio). Tramas más largas estiman mejor la potencia, pero emborronan los transitorios.

En código: sustracción espectral

Python

```python
import numpy as np
from scipy import signal

fs, rng = 8000, np.random.default_rng(1)
t = np.arange(0, 3, 1/fs)
s = np.sin(2*np.pi*440*t)*(t > 1)
x = s + 0.3*rng.standard_normal(t.size)
f, tt, X = signal.stft(x, fs, nperseg=256)
A = abs(X)**2
Pv = A[:, tt < 0.9].mean(axis=1,
                         keepdims=True)
a, b = 2.0, 0.02             # alfa, beta
P = np.maximum(A - a*Pv, b*A)
S = np.sqrt(P)*np.exp(1j*np.angle(X))
_, y = signal.istft(S, fs, nperseg=256)
y = y[:x.size]
snr = lambda z: 10*np.log10(
    np.sum(s**2)/np.sum((z - s)**2))
print(snr(x), snr(y))        # dB
```

Matlab

```matlab
fs = 8000; t = 0:1/fs:3-1/fs;
s = sin(2*pi*440*t).*(t > 1);
x = s + 0.3*randn(size(t));
L = 256; R = L/2;
w = sqrt(hann(L, 'periodic'));
M = floor((numel(x) - L)/R) + 1;
X = zeros(L, M);
for m = 1:M                    % STFT
  X(:, m) = fft(w.*x((m-1)*R+(1:L)).');
end
A = abs(X).^2;
Pv = mean(A(:, 1:60), 2);      % ruido
a = 2; b = 0.02;               % alfa, beta
P = max(A - a*Pv, b*A);
S = sqrt(P).*exp(1j*angle(X));
y = zeros(size(x));
for m = 1:M                    % ISTFT (OLA)
  i = (m-1)*R + (1:L);
  y(i) = y(i) + (w.*real(ifft(S(:, m)))).';
end
snr = @(z) 10*log10(sum(s.^2)/sum((z-s).^2));
disp([snr(x), snr(y)])         % dB
```

Tono de 440 Hz tras 1 s de silencio y ruido de $\sigma = 0{,}3$: la SNR pasa de unos 6 a unos 14 dB.